

Internship project

June 19, 2026

```
[1]: import pandas as pd
import numpy as np
```

```
[2]: df=pd.read_csv("Dataset1.csv")
```

```
[3]: #Numbers of rows and column
print("shape:",df.shape)
```

shape: (186074, 12)

```
[4]: #column names
print(df.columns)
```

```
Index(['SN', 'Train_No', 'Station_Code', '1A', '2A', '3A', 'SL',
       'Station_Name', 'Route_Number', 'Arrival_time', 'Departure_Time',
       'Distance'],
      dtype='object')
```

```
[5]: # dataset information
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 186074 entries, 0 to 186073
Data columns (total 12 columns):
#   Column          Non-Null Count  Dtype
---  -
0   SN              186074 non-null  int64
1   Train_No       186074 non-null  int64
2   Station_Code   186074 non-null  object
3   1A             186074 non-null  int64
4   2A             186074 non-null  int64
5   3A             186074 non-null  int64
6   SL             186074 non-null  int64
7   Station_Name   186074 non-null  object
8   Route_Number   186074 non-null  int64
9   Arrival_time   186074 non-null  object
10  Departure_Time 186074 non-null  object
11  Distance       186074 non-null  int64
```

```
dtypes: int64(8), object(4)
memory usage: 17.0+ MB
```

```
[6]: #first 5 records
df.head()
```

```
[6]:
```

	SN	Train_No	Station_Code	1A	2A	3A	SL	Station_Name	Route_Number	\
0	1	107	SWV	100	100	100	100	SAWANTWADI R	1	
1	2	107	THVM	260	228	196	164	THIVIM	1	
2	3	107	KRMI	345	296	247	198	KARMALI	1	
3	4	107	MAO	490	412	334	256	MADGOAN JN.	1	
4	1	108	MAO	100	100	100	100	MADGOAN JN.	1	

	Arrival_time	Departure_Time	Distance
0	00:00:00	10:25:00	0
1	11:06:00	11:08:00	32
2	11:28:00	11:30:00	49
3	12:10:00	00:00:00	78
4	00:00:00	20:30:00	0

```
[7]: #list of all trains with starting and ending stations
train_routes = df.groupby(['Train_No']).agg(
    Start_Station=('Station_Name', 'first'),
    End_Station=('Station_Name', 'last')
).reset_index()

print(train_routes)
```

	Train_No	Start_Station	End_Station
0	107	SAWANTWADI R	MADGOAN JN.
1	108	MADGOAN JN.	SAWANTWADI R
2	128	MADGOAN JN.	CHHATRAPATI
3	290	DELHI-SAFDAR	DELHI-SAFDAR
4	401	AURANGABAD	VARANASI JN.
...	...	...	...
11108	99904	PUNE JN.	TALEGAON
11109	99905	TALEGAON	SHIVAJINAGAR
11110	99906	PUNE JN.	TALEGAON
11111	99907	TALEGAON	PUNE JN.
11112	99908	PUNE JN.	TALEGAON

```
[11113 rows x 3 columns]
```

```
[8]: #Numbers of stops per train
stops_per_train = df.groupby(
    ['Train_No']
)['Station_Name'].count().reset_index()
```

```
stops_per_train.rename(
    columns={'Station_Name':'Total_Stops'},
    inplace=True
)

print(stops_per_train)
```

	Train_No	Total_Stops
0	107	4
1	108	4
2	128	22
3	290	14
4	401	12
...	...	...
11108	99904	12
11109	99905	11
11110	99906	12
11111	99907	12
11112	99908	12

[11113 rows x 2 columns]

```
[9]: #Trains with maximum and minimum stops
max_stop_train = stops_per_train.loc[
    stops_per_train['Total_Stops'].idxmax()
]

min_stop_train = stops_per_train.loc[
    stops_per_train['Total_Stops'].idxmin()
]

print("Maximum Stops Train:")
print(max_stop_train)

print("\nMinimum Stops Train:")
print(min_stop_train)
```

Maximum Stops Train:
Train_No 53041
Total_Stops 118
Name: 5977, dtype: int64

Minimum Stops Train:
Train_No 557
Total_Stops 2
Name: 11, dtype: int64

```
[10]: #convert time columns
import pandas as pd
df['Arrival_time'] = pd.to_datetime(
    df['Arrival_time'],
    format='%H:%M:%S',
    errors='coerce'
)

df['Departure_Time'] = pd.to_datetime(
    df['Departure_Time'],
    format='%H:%M:%S',
    errors='coerce'
)
```

```
[11]: #Check missing values
df[['Arrival_time', 'Departure_Time']].isnull().sum()
```

```
[11]: Arrival_time      0
      Departure_Time    0
      dtype: int64
```

```
[12]: #Get first and last station time
journey = df.groupby(
    ['Train_No']
).agg(
    start_time=('Departure_Time', 'first'),
    end_time=('Arrival_time', 'last')
).reset_index()
```

```
[13]: #Calculate duration
journey['journey_duration'] = (
    journey['end_time'] -
    journey['start_time']
)
```

```
[14]: journey['duration_hours'] = (
    journey['journey_duration']
    .dt.total_seconds()/3600
)
```

```
[15]: journey.head(500)
```

```
[15]:   Train_No      start_time      end_time  journey_duration \
0      107 1900-01-01 10:25:00 1900-01-01 12:10:00  0 days 01:45:00
1      108 1900-01-01 20:30:00 1900-01-01 22:25:00  0 days 01:55:00
2      128 1900-01-01 19:40:00 1900-01-01 17:45:00 -1 days +22:05:00
3      290 1900-01-01 18:30:00 1900-01-01 02:30:00 -1 days +08:00:00
```

```

4          401 1900-01-01 21:30:00 1900-01-01 10:00:00 -1 days +12:30:00
..          ...
495        12067 1900-01-01 06:30:00 1900-01-01 13:20:00 0 days 06:50:00
496        12068 1900-01-01 14:30:00 1900-01-01 21:20:00 0 days 06:50:00
497        12069 1900-01-01 06:25:00 1900-01-01 13:25:00 0 days 07:00:00
498        12070 1900-01-01 15:00:00 1900-01-01 22:00:00 0 days 07:00:00
499        12071 1900-01-01 14:00:00 1900-01-01 21:30:00 0 days 07:30:00

```

```

duration_hours
0          1.750000
1          1.916667
2         -1.916667
3         -16.000000
4         -11.500000
..          ...
495         6.833333
496         6.833333
497         7.000000
498         7.000000
499         7.500000

```

[500 rows x 5 columns]

```

[16]: #classify routes
def classify_route(hours):

    if hours < 6:
        return 'Short'

    elif hours <= 12:
        return 'Medium'

    else:
        return 'Long'

journey['route_type'] = journey[
    'duration_hours'
].apply(classify_route)

```

```

[17]: journey['route_type'].value_counts()

```

```

[17]: route_type
Short      9973
Medium     977
Long       163
Name: count, dtype: int64

```

```
[18]: #station wise train frequency
station_freq = df.groupby(
    'Station_Name'
)['Train_No'].nunique().reset_index()

station_freq.columns = [
    'Station_Name',
    'Train_count'
]

station_freq.sort_values(
    'Train_count',
    ascending=False,
    inplace=True
)

station_freq.head(10)
```

```
[18]:      Station_Name  Train_count
1755      CST-MUMBAI         1027
3466      KALYAN JN          828
7509         THANE          796
6834      SEALDAH          745
1592  CHENNAI BEAC          738
2950    HOWRAH JN.          699
1773         DADAR          598
2213    DUM DUM JN.          463
4245         KURLA          462
7401    TAMBARAM          434
```

```
[19]: print(
    station_freq.nlargest(
        10,
        'Train_count'
    )
)
```

```
      Station_Name  Train_count
1755      CST-MUMBAI         1027
3466      KALYAN JN          828
7509         THANE          796
6834      SEALDAH          745
1592  CHENNAI BEAC          738
2950    HOWRAH JN.          699
1773         DADAR          598
2213    DUM DUM JN.          463
4245         KURLA          462
```

7401 TAMBARAM 434

```
[20]: #Data quality check
df.isnull().sum()
```

```
[20]: SN                0
      Train_No         0
      Station_Code     0
      1A               0
      2A               0
      3A               0
      SL              0
      Station_Name     0
      Route_Number     0
      Arrival_time     0
      Departure_Time   0
      Distance         0
      dtype: int64
```

```
[21]: df[df['Arrival_time'].isnull()]
```

```
[21]: Empty DataFrame
      Columns: [SN, Train_No, Station_Code, 1A, 2A, 3A, SL, Station_Name,
      Route_Number, Arrival_time, Departure_Time, Distance]
      Index: []
```

```
[22]: #check duplicates records
duplicates = df.duplicated()

print("Duplicate Records:", duplicates.sum())
```

Duplicate Records: 0

```
[23]: invalid_distance_routes = []

for train, group in df.groupby('Train_No'):

    if not group['Distance'].is_monotonic_increasing:
        invalid_distance_routes.append(train)

print(invalid_distance_routes)
```

```
[]
```

```
[24]: df[df['Train_No'].isin(invalid_distance_routes)]
```

```
[24]: Empty DataFrame
      Columns: [SN, Train_No, Station_Code, 1A, 2A, 3A, SL, Station_Name,
```

```
Route_Number, Arrival_time, Departure_Time, Distance]
Index: []
```

```
[25]: #save clean dataset
df.to_csv(
    'verified_Dataset1.csv',
    index=False
)
```

```
[26]: #average journey duration
avg_duration = journey.groupby(
    'route_type'
)['duration_hours'].mean().reset_index()

print(avg_duration)
```

	route_type	duration_hours
0	Long	14.639059
1	Medium	8.064790
2	Short	-0.528544

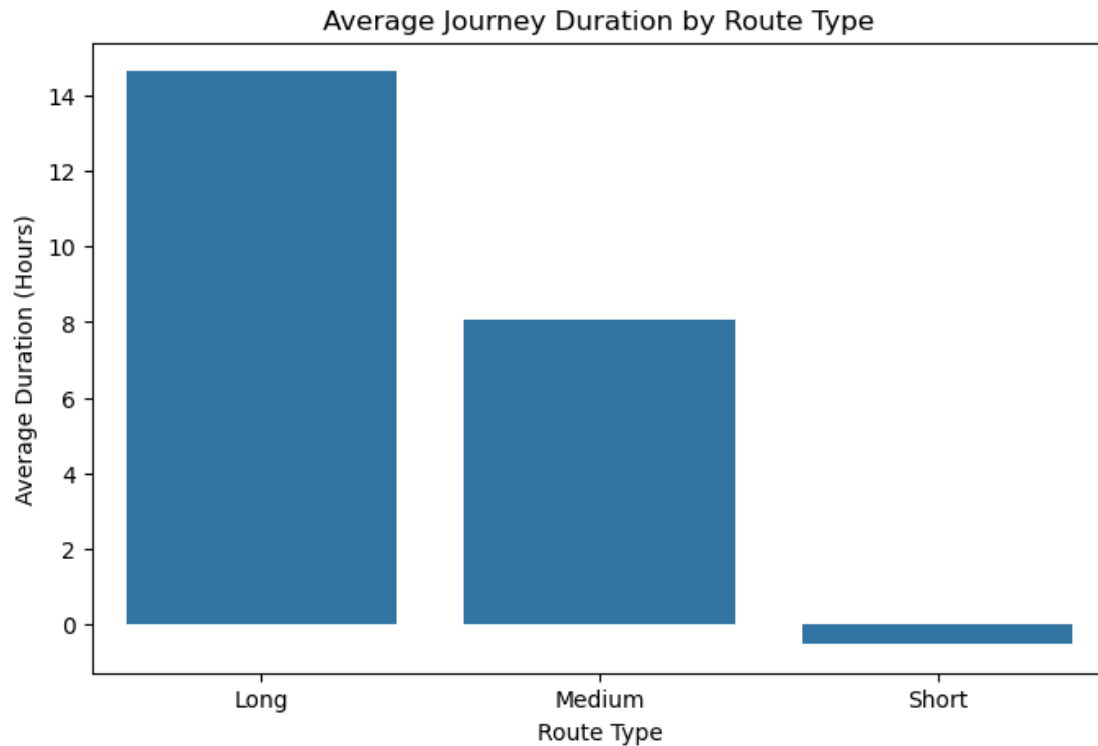
```
[27]: import matplotlib.pyplot as plt
import seaborn as sns

plt.figure(figsize=(8,5))

sns.barplot(
    data=avg_duration,
    x='route_type',
    y='duration_hours'
)

plt.title('Average Journey Duration by Route Type')
plt.xlabel('Route Type')
plt.ylabel('Average Duration (Hours)')

plt.show()
```

```
[28]: #high traffic stations
top_stations = station_freq.nlargest(
    10,
    'Train_count'
)

print(top_stations)
```

	Station_Name	Train_count
1755	CST-MUMBAI	1027
3466	KALYAN JN	828
7509	THANE	796
6834	SEALDAH	745
1592	CHENNAI BEAC	738
2950	HOWRAH JN.	699
1773	DADAR	598
2213	DUM DUM JN.	463
4245	KURLA	462
7401	TAMBARAM	434

```
[29]: plt.figure(figsize=(10,6))

sns.barplot(
```

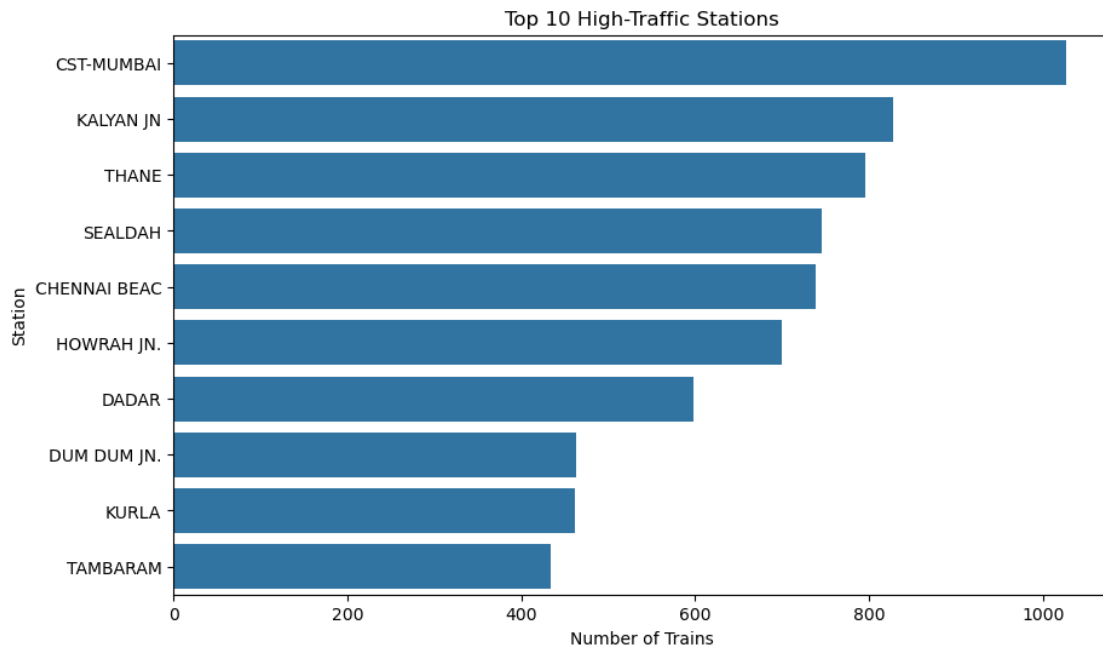
```

data=top_stations,
x='Train_count',
y='Station_Name'
)

plt.title('Top 10 High-Traffic Stations')
plt.xlabel('Number of Trains')
plt.ylabel('Station')

plt.show()

```



```

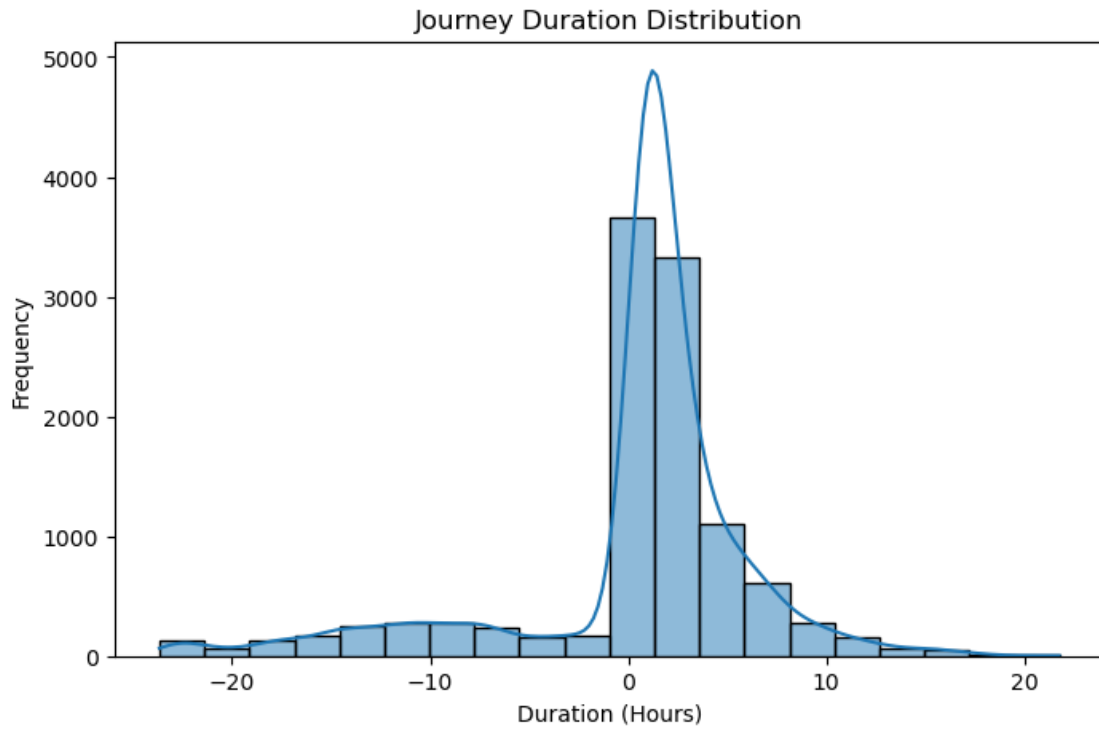
[30]: #Journey duration distribution
plt.figure(figsize=(8,5))

sns.histplot(
    journey['duration_hours'],
    bins=20,
    kde=True
)

plt.title('Journey Duration Distribution')
plt.xlabel('Duration (Hours)')
plt.ylabel('Frequency')

plt.show()

```

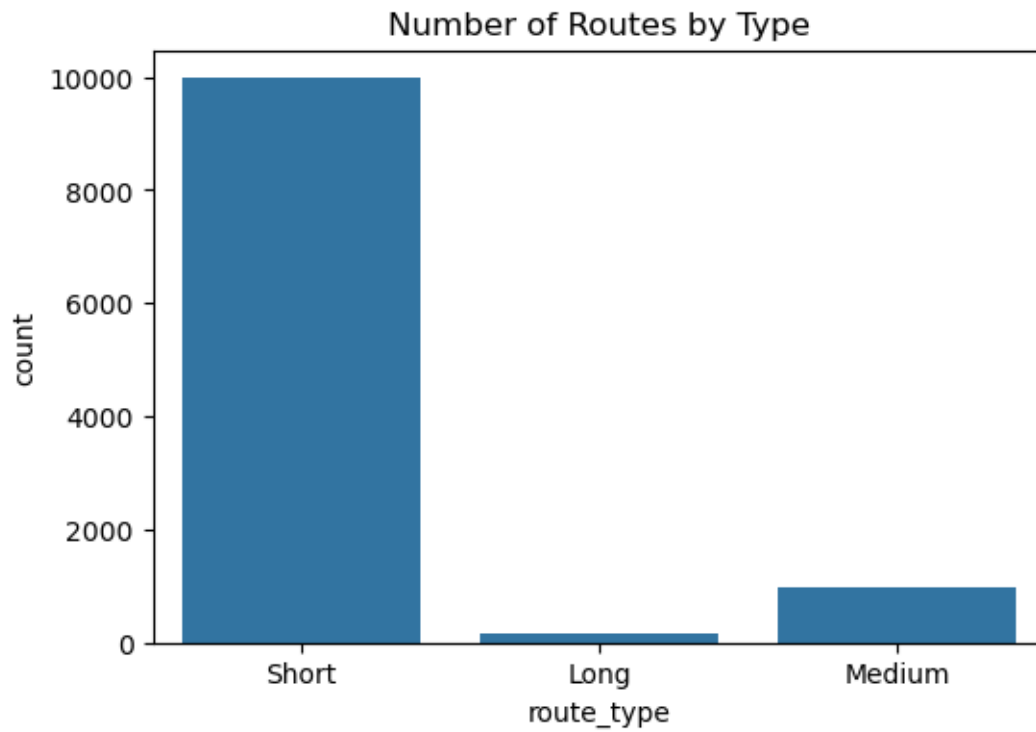


```
[31]: #route type count
plt.figure(figsize=(6,4))

sns.countplot(
    data=journey,
    x='route_type'
)

plt.title('Number of Routes by Type')

plt.show()
```

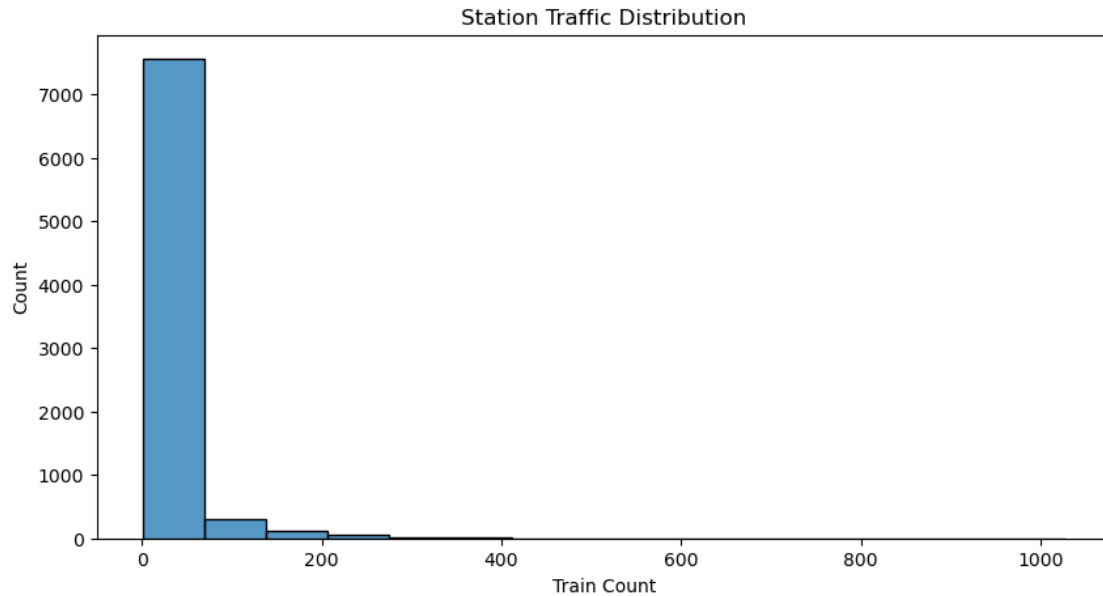


```
[32]: # station traffic distribution
plt.figure(figsize=(10,5))

sns.histplot(
    station_freq['Train_count'],
    bins=15
)

plt.title('Station Traffic Distribution')
plt.xlabel('Train Count')

plt.show()
```



```
[33]: #SUMMARY KRY OBSERVATION
print("Average Duration:")
print(journey['duration_hours'].mean())

print("\nLongest Journey:")
print(journey.loc[
    journey['duration_hours'].idxmax()
])

print("\nShortest Journey:")
print(journey.loc[
    journey['duration_hours'].idxmin()
])

print("\nTop 10 Stations:")
print(top_stations)
```

Average Duration:
0.44941060019796647

Longest Journey:

Train_No	7006
start_time	1900-01-01 01:30:00
end_time	1900-01-01 23:15:00
journey_duration	0 days 21:45:00
duration_hours	21.75
route_type	Long

Name: 165, dtype: object

Shortest Journey:

Train_No 43892
start_time 1900-01-01 23:55:00
end_time 1900-01-01 00:15:00
journey_duration -1 days +00:20:00
duration_hours -23.666667
route_type Short

Name: 5467, dtype: object

Top 10 Stations:

	Station_Name	Train_count
1755	CST-MUMBAI	1027
3466	KALYAN JN	828
7509	THANE	796
6834	SEALDAH	745
1592	CHENNAI BEAC	738
2950	HOWRAH JN.	699
1773	DADAR	598
2213	DUM DUM JN.	463
4245	KURLA	462
7401	TAMBARAM	434

```
[34]: avg_duration_hours = 0.44941060019796647 * 24  
  
print(avg_duration_hours)
```

10.785854404751195

```
[35]: #Pivot tables for station wise train distribution  
pivot_station = pd.pivot_table(  
    df,  
    index='Station_Name',  
    values='Train_No',  
    aggfunc='nunique'  
)  
  
pivot_station = pivot_station.sort_values(  
    by='Train_No',  
    ascending=False  
)  
  
print(pivot_station.head(10))
```

	Train_No
Station_Name	
CST-MUMBAI	1027

KALYAN JN	828
THANE	796
SEALDAH	745
CHENNAI BEAC	738
HOWRAH JN.	699
DADAR	598
DUM DUM JN.	463
KURLA	462
TAMBARAM	434

```
[36]: # top 10 station chart
import matplotlib.pyplot as plt
import seaborn as sns

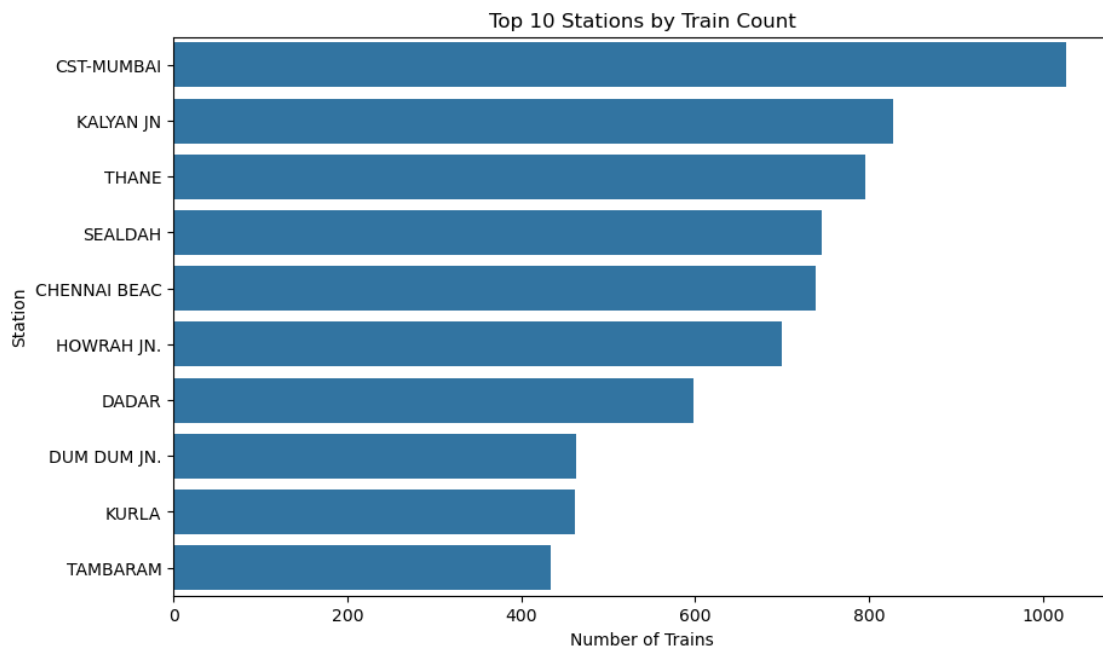
top10 = pivot_station.head(10)

plt.figure(figsize=(10,6))

sns.barplot(
    x=top10['Train_No'],
    y=top10.index
)

plt.title('Top 10 Stations by Train Count')
plt.xlabel('Number of Trains')
plt.ylabel('Station')

plt.show()
```



```
[37]: #cross tab analysis
#route type vs train count
route_crosstab = pd.crosstab(
    journey['route_type'],
    columns='Train Count'
)

print(route_crosstab)
```

col_0	Train Count
route_type	
Long	163
Medium	977
Short	9973

```
[38]: #station vs route type
df_route = df.merge(
    journey[['Train_No', 'route_type']],
    on='Train_No'
)
```

```
[39]: station_route = pd.crosstab(
    df_route['Station_Name'],
    df_route['route_type']
)

print(station_route.head(500))
```

route_type	Long	Medium	Short
Station_Name			
.BAGHAJATIN	0	0	217
ABADA	0	0	90
ABHAIPUR	0	13	49
ABHANPUR JN.	0	0	12
ABHAYAPURI A	0	1	5
...	...	...	...
BAHJOI	0	2	9
BAHMAN DIWAN	0	3	6
BAHMNIWALA	0	0	10
BAHORA CHAND	0	4	3
BAHRAICH	0	9	9

[500 rows x 3 columns]


```
[40]: #visualize cross-tab results
plt.figure(figsize=(12,8))

sns.heatmap(
    station_route.head(20),
    annot=True,
    cmap='YlGnBu'
)

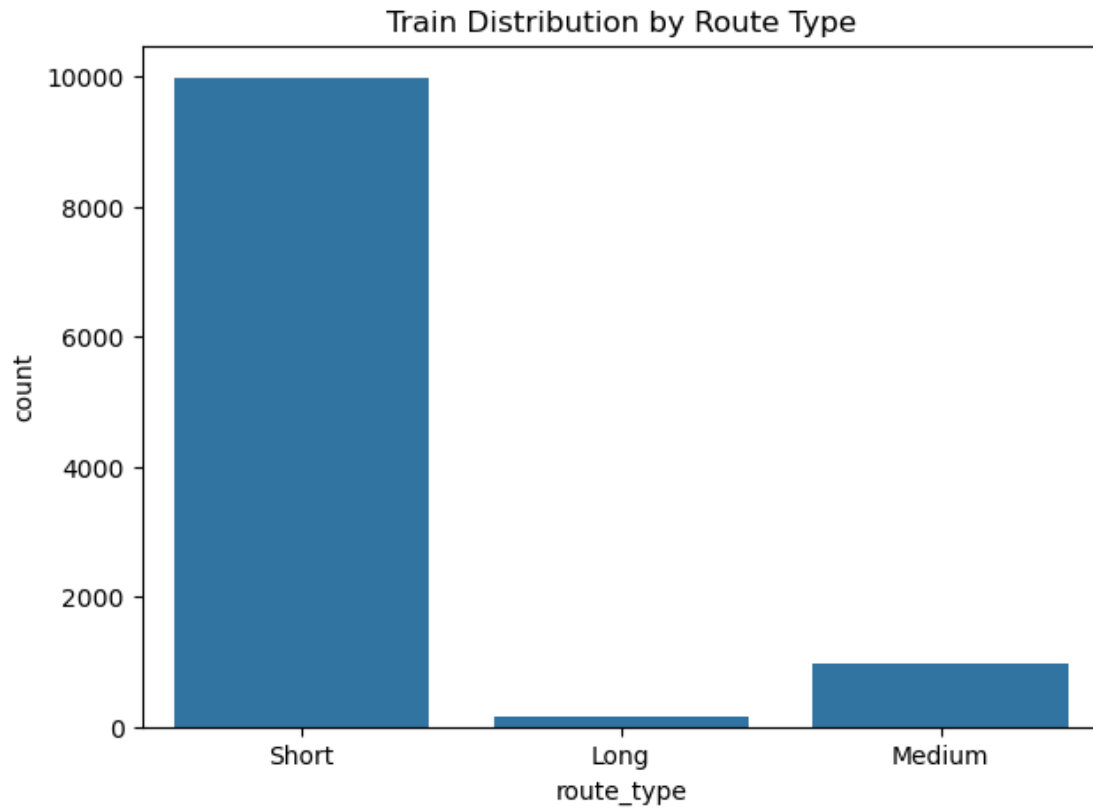
plt.title('Station vs Route Type')
plt.show()
```



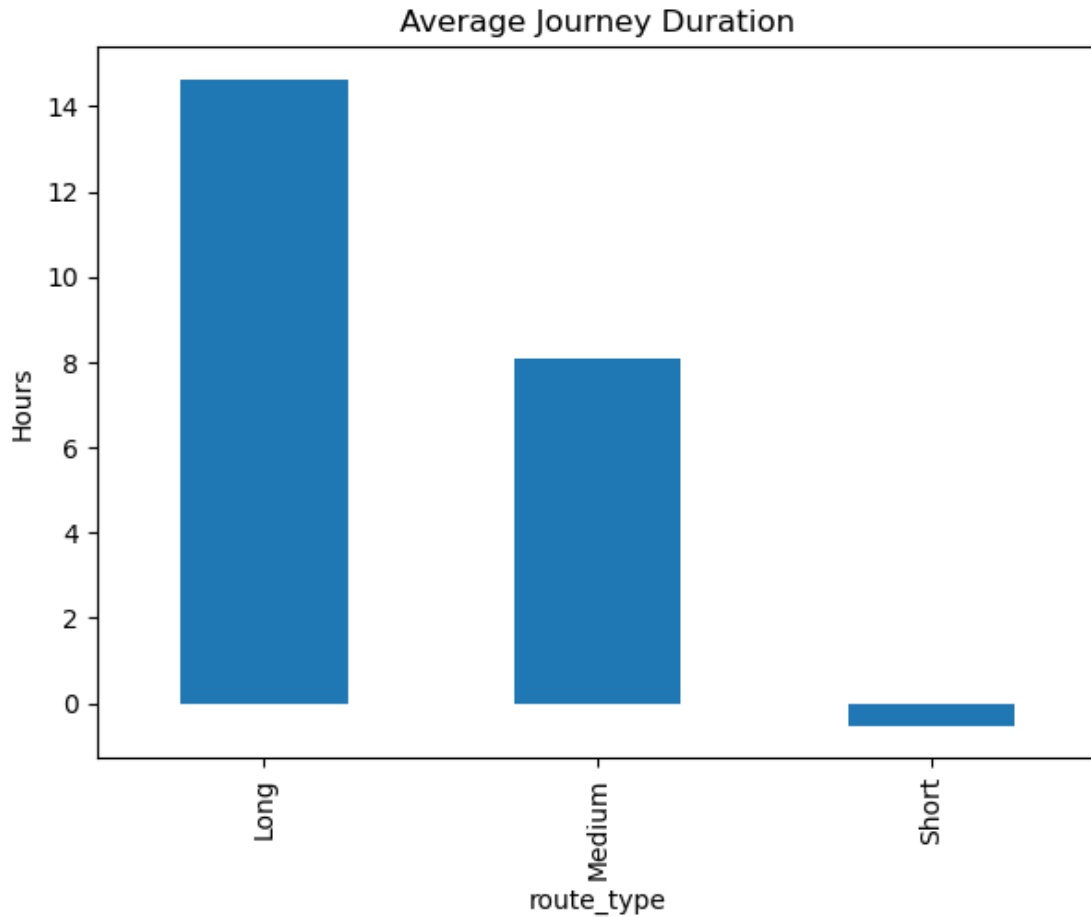
```
[41]: #route type comparision
plt.figure(figsize=(7,5))

sns.countplot(
    data=journey,
    x='route_type'
)

plt.title('Train Distribution by Route Type')
plt.show()
```



```
[42]: avg_route = journey.groupby(  
        'route_type'  
    )['duration_hours'].mean()  
  
    avg_route.plot(  
        kind='bar',  
        figsize=(7,5)  
    )  
  
    plt.title('Average Journey Duration')  
    plt.ylabel('Hours')  
  
    plt.show()
```



```
[43]: #summarize advance insight
print(station_route.describe())

print("\nTop Stations:")
print(pivot_station.head(10))
```

route_type	Long	Medium	Short
count	8099.00000	8099.000000	8099.000000
mean	0.86628	3.146314	18.962341
std	1.69261	5.335330	43.248627
min	0.00000	0.000000	0.000000
25%	0.00000	0.000000	4.000000
50%	0.00000	2.000000	7.000000
75%	1.00000	4.000000	15.000000
max	20.00000	73.000000	1006.000000

Top Stations:

Train_No

Station_Name	
CST-MUMBAI	1027
KALYAN JN	828
THANE	796
SEALDAH	745
CHENNAI BEAC	738
HOWRAH JN.	699
DADAR	598
DUM DUM JN.	463
KURLA	462
TAMBARAM	434

```
[44]: #Train enquiry systems
import pandas as pd

df = pd.read_csv("Dataset1.csv")

df = df.sort_values(
    ['Train_No', 'Station_Name']
)
```

```
[45]: #find direct train function
def find_trains(source, destination):

    results = []

    for train_no, group in df.groupby('Train_No'):

        stations = list(group['Station_Name'])

        if source in stations and destination in stations:

            source_idx = stations.index(source)
            dest_idx = stations.index(destination)

            if source_idx < dest_idx:

                train_no = group[
                    'Train_Name'
                ].iloc[0]

                results.append(
                    [train_no, train_name]
                )

    return results
```

```
[ ]: #calculate journey duration
def get_duration(train_no,
                 source,
                 destination):

    train = df[
        df['Train_No'] == train_no
    ]

    source_row = train[
        train['Station_Name'] == source
    ].iloc[0]

    dest_row = train[
        train['Station_Name'] == destination
    ].iloc[0]

    start = pd.to_datetime(
        source_row['departure_time']
    )

    end = pd.to_datetime(
        dest_row['Arrival_Time']
    )

    duration = end - start

    return duration
```

```
[52]: #user input
source = input(
    "Enter Source Station: "
)

destination = input(
    "Enter Destination Station: "
)
```

Enter Source Station: THIVIM
Enter Destination Station: KARMALI

```
[53]: #display result
trains = find_trains(
    source,
    destination
)
```

```

if len(trains) == 0:

    print(
        "No Direct Train Found"
    )

else:

    print(
        "\nAvailable Direct Trains:\n"
    )

    for train_no in trains:

        duration = get_duration(
            train_no,
            source,
            destination
        )

        print(
            f"Train No: {train_no}"
        )

        print(
            f"Estimated Duration: {duration}"
        )

    print("-"*40)

```

No Direct Train Found

```

[49]: #search by train no
train_no = int(
    input("Enter Train Number: ")
)

df[
    df['Train_No'] == train_no
]

```

Enter Train Number: 108

```

[49]:

```

	SN	Train_No	Station_Code	1A	2A	3A	SL	Station_Name	Route_Number	\
5	2	108	KRMI	265	232	199	166	KARMALI	1	
4	1	108	MAO	100	100	100	100	MADGOAN JN.	1	
7	4	108	SWV	515	432	349	266	SAWANTWADI R	1	

6	3	108	THVM	355	304	253	202	THIVIM	1
---	---	-----	------	-----	-----	-----	-----	--------	---

	Arrival_time	Departure_Time	Distance
5	21:04:00	21:06:00	33
4	00:00:00	20:30:00	0
7	22:25:00	00:00:00	83
6	21:26:00	21:28:00	51

```
[54]: #route display
stations = df[
    df['Train_No'] == train_no
]['Station_Name']

print(list(stations))
```

```
['KARMALI', 'MADGOAN JN.', 'SAWANTWADI R', 'THIVIM']
```

```
[ ]:
```